DAM Price Forecasting – Technical Report

1. Problem Understanding

The objective is to forecast **15-minute electricity prices** in the Day-Ahead Market (DAM) for a full day (96 time blocks). At prediction time for a given day *D*, only information up to **D−1 23:45** is available. This imposes a strict **no-leakage constraint**, disallowing use of intra-day information for the target day.

Key challenges:

- Strong **intraday seasonality** (96-block structure)
- Presence of **price spikes (capped values ~10000)**
- **Non-stationarity** and regime shifts
- High dependence on **previous day patterns (lag-96)**

2. Data Analysis Insights

Exploratory Data Analysis revealed:

◈ **Intraday Structure -** Prices exhibit a consistent **daily pattern**, with predictable peak and valley hours.

◈ **Strong Lag Dependence -** The previous day's price at the same time block (**lag-96**) is a strong baseline predictor.

◈ **Spikes and Heavy Tail -** Price distribution is **right-skewed with extreme spikes**, indicating: non-Gaussian behaviour, difficulty for smooth models

◈ **Buy/Sell Influence**

Buy/sell quantities doesn't correlate much with price, and are less stable than lag features.

3. Feature Engineering

Feature engineering was designed under strict **no-leakage constraints**. Design principle: prioritize string signals, simple transformations, no leakage.

◈ Time Features – Capture periodic structure like hour, block, sin/cos encoding of the week, weekend indicators etc

◈ Lag Features (Core Signal) – price_lag_96, 192, 672 etc, form strongest predictive signal

◈ Trend and Deviation – like daily difference, rolling mean, capture short term directional changes

◈ Volatility Features like rolling std, previous-day range, designed to help the models recognize spike prone times

◈ Market Imbalance – adds domain-specific signal, reflecting demand-supply mismatch

4. Baselines

◈ Naive Model: price(t) = price(t-96). This is a strong benchmark in this forecasting

◈ Statistical Models: ARIMA, SARIMA, Prophet, fail on spikes and underperform naïve baseline

5. Model Architecture

Tree-based ensemble models were used: Random Forest, LightGBM, CatBoost , XGBoost. We did direct multi -step forecasting using the tree models, so there is no error accumulation.

Why tree models?

- Handle non linear relationships
- Robust to outliers
- No scaling required
- Capture interactions between lag and time features

Used hyper parameter tuning on validation splits to find the best hyperparameters for training each tree model

Also the visualizations of validation outputs helped in iteratively adding specific features (built spike based features after seeing the plots)

Used the validation results to gauge the weights of each prediction in final ensemble model.

Final models are trained on train and validation data and predictions for the test data were recorded

6. Evaluation

Time based split: Train (Apr–Aug), Val (Sep), Test (Oct).

Metrics: MAE, RMSE, MAPE.

7. Model Comparison

- These models fared well in RMSE when compared to naïve. Shows that they work relatively well during uncertainty

- All ML models struggled to beat naive MAE baseline consistently

- Random Forest performed best among individual models

- The classical time series models do recursive forecasting (unlike tree models here), and the error accumulation is one of the reasons for their reduced performance

8. Ensemble

Naïve provides strong baseline and tree models capture. Tree models capture nonlinear deviations. So final answer is weighted combination of naïve + tree models. Right now, it is hardcoded based on the validation results – which can be further worked on.

9. Results

Model	**MAE**	**RMSE**	**MAPE**
ARIMA	1591	2710	36%
SARIMA	1364	1946	25%
Prophet	2845	3420	50%
Naive	876	1685	17%
RF	902	1396	16.1%
LGB	941	1407	16.2%
CAT	978	1419	17.3%
XGB	946	1422	16.5%
Ensemble	870	1454	15.9%

10. Learnings

Naive is strong, spikes are hard, ensemble helps, and strict time-series discipline is critical. Spikes are the hardest part. More features are not always better, especially because

boosting models can overfit. It is crucial and difficult to look for data leakage in time series when splitting or building features

10. Conclusion

Final system combines feature engineering, tree models, and ensemble for stable performance. Further steps would be strategic ensemble of the model. Using deep learning models to capture more complex relationships, probabilistic forecasting, spike detection etc.